

What a Product Costs

The same multiplications and additions, counted, and then measured in operations per second: the numbers differ by factors of thousands.

Count the operations

The innermost line of the three loops of Act 1, `total += W[j, c] * X[i, c]`, carries out one multiplication and one addition. The loops pass through it once for every point, every neuron and every coordinate:

$$n \times K \times m$$

- **Predict:** when n is doubled, by what factor does the time grow? When n , m and K are all doubled?
- Measured on random numbers: $n = 400$ points, $m = 30$ coordinates, $K = 20$ neurons; then n doubled; then all three doubled.

n	m	K	$n \times K \times m$	seconds	ratio to the first	nanoseconds per pass
400	30	20	240,000	0.037	1.0	153
800	30	20	480,000	0.076	2.1	158
800	60	40	1,920,000	0.298	8.1	155

$O(nKm)$

- The time grows as the number of passes does: a factor near 2 when n doubles, near 8 when all three double. The time of one pass is nearly the same in the three rows: **time $\approx$ (time of one pass) $\times n \times K \times m$.**
- Computer science writes **$O(nKm)$** , read "of the order of $n K m$ ": the notation keeps how the count grows with the sizes, and drops the constant in front, the time of one pass.
- For a layer with $n = m = K$ the count is n^3 : double the size, and the time is multiplied by 8. **That is the factor of Act 2 between the sizes 2,048 and 4,096.**
- The page **What matrix operations cost** lists the count for every operation of the meeting. A transpose and a reshape cost nothing: no number is moved.

ikoutis.github.io/cs301-labs/unit-2/pages/matrix-costs.html

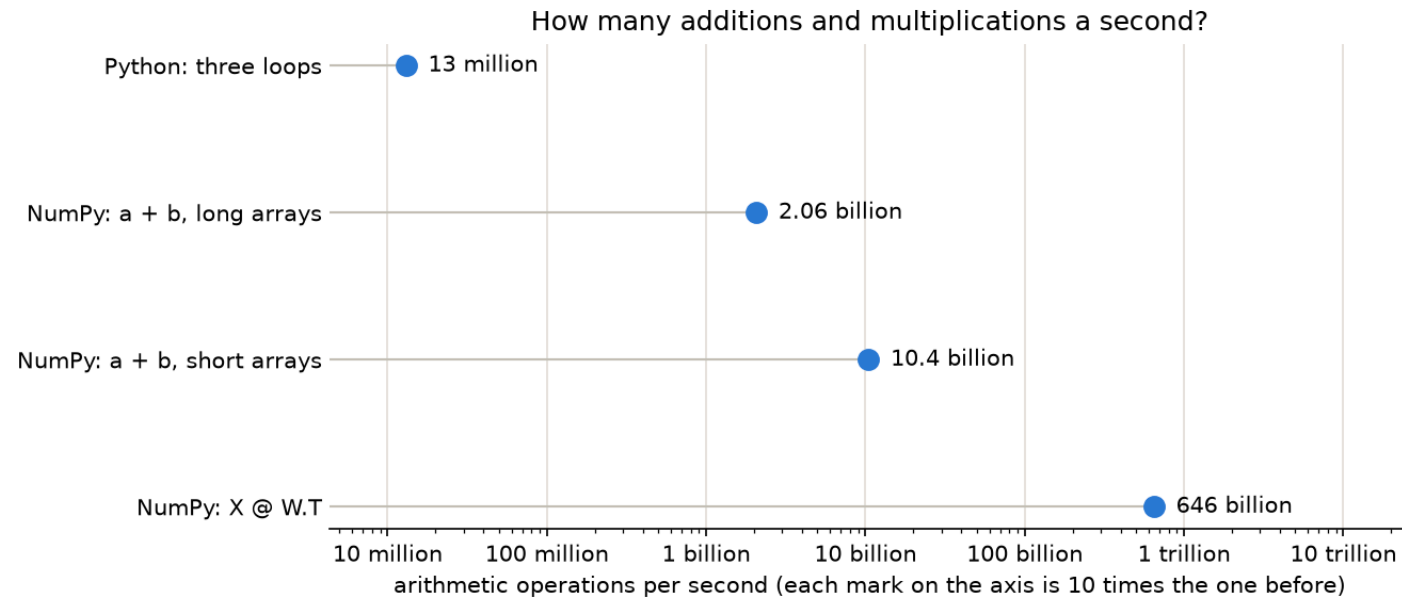
The same operations, in other seconds

The count says how the time **grows**, not how long one operation takes. That is measured as **arithmetic operations per second**, each multiplication and each addition counting as one, for five ways of computing:

- **Python, three loops:** the first measurement above.
- **NumPy, `a + b` on long arrays:** two arrays of 16 million numbers, added four times.
- **NumPy, `a + b` on short arrays:** two arrays of 16 thousand numbers, added 4,096 times: the same number of additions.
- **NumPy, the product `X @ W.T`** with $n = m = K = 2,048$.
- **The GPU, the same product** at size 4,096: from your measurement of Act 2.

way of computing	operations per second	ratio to the slowest
Python: three loops	13 million	1
NumPy: <code>a + b</code> , long arrays	2.06 billion	158
NumPy: <code>a + b</code> , short arrays	10.4 billion	800
NumPy: <code>X @ W.T</code>	646 billion	49,536

How many additions and multiplications a second?



- Each mark on the axis is ten times the one before. Four ways of computing, on the machine that built these slides; **the fifth mark, the GPU, is yours to measure.**
- Python against NumPy: the interpreter's work around each operation, which meeting 10 measured.

Long arrays against short arrays: memory-bound

- The two rows run **the same additions, by the same code**. Only the place of the numbers differs.
- A processor has a small, fast memory of its own, the **cache**. The short arrays fit into it and stay there.
- The long arrays do not fit: each number is fetched from the main memory, used in **one** addition, and the result is sent back. The processor spends most of its time waiting for numbers.
- Such a computation is **memory-bound**: the memory sets its speed, not the arithmetic.

The page **CPU and GPU**, parts 3 and 4: one product divided among the cores, and the two memories.

The product: compute-bound

- In `X @ W.T` a number is not used once: each entry of X is multiplied with a weight of **every** neuron.
- NumPy's product works on blocks of the two matrices that fit into the cache, and uses each number many times while it is there. The processor spends its time on arithmetic: **compute-bound**. Every core helps, and so do the instructions that handle several numbers at once.
- **The GPU:** thousands of small cores on the same product. This pays for compute-bound work on large arrays, and, as Act 2 measured, not for small ones.
- $O()$ is the right first estimate of a cost. The chart shows what it leaves out: how the program is run, where the numbers are, and which processor does the work.

Predict the time of a layer you cannot try

$$\text{time} \approx \frac{\text{number of operations}}{\text{operations per second}}$$

A layer of $K = 1,000$ neurons on $n = 60,000$ grey images of 28×28 pixels, so $m = 784$: $2 \times n \times K \times m$ operations.

<code>n = 60,000, m = 784, K = 1,000:</code>	<code>94,080,000,000 operations (94.1 billion)</code>
way of computing	predicted time
Python: three loops	2 hours
NumPy: <code>X @ W.T</code>	146 milliseconds

- The times belong to the same computation, with the same $O()$. Review Quiz 11 asks for such a prediction for another layer, and which of the measurements of this act is memory-bound and which compute-bound.

Now in Colab: Act 3

- **Act 3** of the class notebook: predict the two factors, then run the cell that measures the loops and the rates. The GPU's rate comes from your measurement of Act 2, in the same notebook.
- Change the three sizes of the layer in the last cell to a layer of your own, and read the predicted times.
- **Post**, as a reply to your group's first post: the printed line, the chart with one sentence on what it shows, and one sentence on the two rows that your group finds most surprising. One member posts; begin with the names of all the members.

What we saw today

	one point x	n points X
one neuron: w, b	$w @ x + b$	$X @ w + b$
K neurons: $W (K, m), b (K,)$	$W @ x + b$	$X @ W.T + b$

- The same line in NumPy and in PyTorch, on the CPU and on a GPU. A layer of neurons is a matrix with one neuron in each row.
- The product costs $O(nKm)$ operations. What a second of computing is worth depends on how the program is run, where the numbers are, and which processor does the work.
- **After class:** the full notebooks of the three acts, linked on the meeting's page, and Review Quiz 11, which opens now.